PROJECT DOCUMENTATION

FaceAttend

Face recognition attendance system with liveness (anti-photo) protection, a kiosk camera screen and an admin console. End-to-end guide: how it works, every module, where it is used, and how to run it.

Version: **hardening** branch · 28 Sep 2026 · Stack: React + Node.js/Express + MongoDB + TensorFlow.js

1. Quick facts
2. How to run
3. Architecture (big picture)
4. Workflow A — first-time setup
5. Workflow B — register a person
6. Workflow C — mark attendance (kiosk)
7. Workflow D — admin daily work
8. Folder map
9. Backend modules (what + where used)
10. Frontend modules (what + where used)
11. Libraries / packages used
12. API endpoints
13. Database collections
14. Important settings (.env)
15. Security in short
16. Messages & troubleshooting
17. Commands cheat-sheet
18. Remember it in 60 seconds

1. Quick facts

What it does
A camera screen (kiosk) recognizes a registered person, checks they are a **real live person** (not a photo/phone), and records **one attendance per person per day**.

Who uses it
Person — stands in front of the kiosk.
Admin — registers people, views/export attendance, manages kiosks.

Part	Technology	Folder	Runs on
Frontend (website)	React 19, react-router 7	`frontend/`	http://localhost:3000
Backend (API)	Node.js, Express 5	`backend/`	http://localhost:5000
Database	MongoDB (Mongoose 8)	—	mongodb://127.0.0.1:27017/faceattendance
Face AI	@vladmandic/face-api on TensorFlow.js (WASM)	`backend/src/services`	inside the backend
Anti-spoof AI	MiniFASNet ONNX models + onnxruntime-node	`backend/models/antispoof`	inside the backend

The 4 AI steps (remember "D-E-M-L")

D Detect
Find faces in the frame (SSD MobileNet v1) + 68 face landmarks.

E Embed
Turn the face into 128 numbers ("face fingerprint").

M Match
Compare with registered people. Distance must be small **and** clearly better than 2nd best.

L Liveness
Anti-photo model + "turn your head" challenge.

Golden rules of this project

- **Unsure = do not mark.** "Unable to verify identity" is always better than marking the wrong person.
- **Server decides everything.** The browser only shows the camera and messages.
- **No face photos are stored.** Only encrypted 128-number templates.
- **One record per person per day** — guaranteed by the database (unique index).

2. How to run

Before you start: MongoDB must be running (on Windows it runs as a service automatically).

Terminal 1 — Backend

```
cd C:\Users\vicus\Desktop\facedetection\backend
npm start
```

Wait for: `Server listening, port 5000`

Terminal 2 — Frontend

```
cd C:\Users\vicus\Desktop\facedetection\frontend
npm start
```

Wait for: `Compiled successfully`

Then open **http://localhost:3000**. Stop a server with **Ctrl + C**.

Page	URL	Who
Home	http://localhost:3000	Everyone
Admin console	http://localhost:3000/admin	Admin (username `admin` + your password)
Kiosk (camera)	http://localhost:3000/kiosk	Activated device

One-time setup (new computer / fresh clone)

```
cd backend
npm install
copy .env.example .env          # fill JWT_SECRET and BIOMETRIC_KEY (commands inside the file)
npm run create-admin -- admin   # asks for a password (10+ chars, letters + numbers)
cd ..\frontend
npm install
```

Production (single server)

```
cd frontend  &&  npm run build
cd ..\backend  &&  set NODE_ENV=production  &&  npm start     # everything on :5000 (use HTTPS)
```

Browsers allow the camera only on **HTTPS** or **localhost**.

3. Architecture (big picture)

Browser (React)

Kiosk page captures a camera frame every ~0.3 s and sends it. Admin pages show data.

frontend/src

⇄

Backend (Express)

Checks login, validates input, runs face AI, liveness, matching, and attendance rules.

backend/src

⇄

MongoDB

students (encrypted face templates), attendances, admins, devices, auditlogs.

faceattendance DB

Request path inside the backend (every API call)

server.js → app.js → helmet / CORS → rateLimits → security (CSRF) → auth (who are you) → routes/* → validate (zod) → services/* → models/* → MongoDB

Any error on the way goes to **errorHandler**, which returns a friendly message and logs the technical details.

Face pipeline for one camera frame

upload (memory) → imageDecoder → faceEngine.detect → count faces = 1? → quality → liveness.passiveScore → faceEngine.descriptor → gallery.match → scanSession (decide) → attendanceService

Startup order (server.js)

1. config (read .env) → 2. db.connect → 3. migrations → 4. faceEngine.init + liveness.init → 5. gallery.load → 6. listen :5000

4. Workflow A — first-time setup

1. **Install & configure**
   npm install in both folders; create `backend/.env` from `.env.example`.
   Modules: config/index.js reads .env

2. **Create the admin account**
   `npm run create-admin -- admin`
   Modules: scripts/create-admin.js → models/Admin.js (password hashed with bcrypt)

3. **Start backend + frontend**
   On start the backend migrates old data, loads AI models and the face gallery.
   Modules: server.js, db.js, migrations/index.js, faceEngine.js, liveness.js, gallery.js

4. **Sign in**
   Open /admin → username + password.
   Frontend: Login.js, AuthContext.js → API POST /api/auth/login → routes/auth.js → middleware/auth.js sets the session cookie

5. **Clean old data**
   Users → "Possible duplicates" (same person registered many times) and "Needs re-registration".
   Frontend: Students.js → GET /api/students/duplicates → gallery.suspectedDuplicates

6. **Activate the kiosk device**
   On the kiosk PC: sign in, open /kiosk, click **Activate this device**. The device gets its own token (revocable in Settings).
   Frontend: Kiosk.js → POST /api/auth/kiosk/activate → models/Device.js

5. Workflow B — register a person

Admin console → **Register user**. Page: `pages/admin/RegisterStudent.js`.

1. **Enter details**
   Full name + employee/roll number. Checked instantly in the browser and again on the server.
   Frontend: utils/format.js (validateName, validateRollNo) · Backend: routes/schemas.js

2. **Guided face capture (5 photos)**
   Look straight → turn slightly left → turn slightly right → straight → straight.
   Each live frame is sent to the server; a photo is auto-captured after **2 good frames in a row**.
   Frontend: components/FaceCapture.js, hooks/useCamera.js → POST /api/students/check-sample → enrollmentService.checkSample → faceAnalysis → quality.checkPose, liveness

3. **Live guidance messages**
   "Move closer to the camera", "Improve lighting", "Please look directly at the camera", "Only one person should be visible"...
   Backend: services/quality.js (MESSAGES)

4. **Review & save**
   All photos are sent together. Server re-checks every photo, makes sure all are the same person, and makes 128-number templates.
   POST /api/students → routes/students.js → enrollmentService.buildSamples → matcher.maxPairwise

5. **Duplicate-face check**
   If this face already belongs to another registered person → warning "*A similar face is already registered. Please verify before continuing.*" Admin can cancel or confirm (logged in audit).
   gallery.findSimilar (DUPLICATE_FACE_THRESHOLD) · audit.js

6. **Encrypt & store**
   Templates encrypted with AES-256-GCM and saved. **Photos are thrown away.** Gallery reloads.
   utils/biometricCrypto.js → models/Student.js → gallery.invalidate

7. **Done**
   "Face registered successfully."

Re-register a face later: Users → open person → "Re-register face" (PUT /api/students/:id/face). Same checks, replaces old templates.

6. Workflow C — mark attendance (kiosk)

Page `pages/Kiosk.js` sends a frame every ~0.3 s to `POST /api/kiosk/scan`. The server keeps a **scan session** (`services/scanSession.js`) — a state machine that decides step by step.

1. **Looking for face…**
   No face → keep scanning. More than one face → "*Please ensure only one person is visible.*" and progress resets.
   faceAnalysis.analyzeFrame → faceEngine.detect
2. **Quality check**
   Too small / blurry / dark / bright / turned → guidance message, frame not counted.
   quality.assessQuality · faceGeometry.headPose
3. **Passive liveness (anti-photo)**
   Two MiniFASNet models score each good frame (0–1). Mean of 3+ frames must be ≥ 0.6. Two frames < 0.2 → "*Unable to confirm a live person.*"
   liveness.passiveScore · faceEngine.antispoofInput
4. **Verifying identity…**
   128-number template compared with everyone. Accept only if distance ≤ 0.5 **and** 2nd-best person is ≥ 0.06 further. Same person on **3 frames in a row** (5 if not high-confidence). Unknown 4 times → "*Face not recognized.*"
   faceEngine.descriptor → gallery.match → matcher.match
5. **Challenge**
   Random: "*Please turn your head slightly to your left/right.*" (screen is mirrored: your left = screen left). Then "*Now look back at the camera.*" 10 s limit.
   liveness.Challenge · randomChallengeType
6. **Confirm**
   One more straight frame must match the **same** person. A different person at any time → "*Unable to verify identity.*"
   scanSession (CONFIRM phase)
7. **Record attendance**
   Insert one record for today (time zone APP_TIMEZONE). If already there → "*Attendance already marked today.*" Late after LATE_AFTER time (optional).
   attendanceService.markAttendance → models/Attendance.js (unique studentId + dayKey) · utils/time.js
8. **Show result, then continue**
   Green card: name, ID, time, status. After ~4 s the kiosk scans for the next person.
   Kiosk.js (result card)

Kiosk colour	Meaning
Grey	Looking for face
Blue	Verifying / checking liveness (progress bar)
Amber	Fix something (lighting, distance, one person only)
Violet	Do the challenge (arrow shows direction)
Green	Attendance marked
Red	Not recognized / not live / failed — nothing recorded

7. Workflow D — admin daily work

Task	Where (page)	API	Backend module
See today's numbers	Dashboard.js	GET /api/admin/dashboard	routes/admin.js
Search / filter users	Students.js	GET /api/students	routes/students.js
Find duplicate faces	Students.js (tab)	GET /api/students/duplicates	gallery.suspectedDuplicates
Edit name / ID	StudentDetail.js	PATCH /api/students/:id	routes/students.js + audit
Activate / deactivate user	StudentDetail.js	PATCH /api/students/:id	gallery.invalidate
Delete user (keeps history)	StudentDetail.js	DELETE /api/students/:id	routes/students.js + audit
View attendance (date, search, status, pages)	Attendance.js	GET /api/attendance	routes/attendance.js
Change Present / Late	Attendance.js	PATCH /api/attendance/:id	routes/attendance.js + audit
Delete a record	Attendance.js	DELETE /api/attendance/:id	routes/attendance.js + audit
Export Excel	Attendance.js	GET /api/attendance/export	exportHandler (ExcelJS)
Change password	Settings.js	POST /api/auth/change-password	routes/auth.js
Revoke a kiosk	Settings.js	PATCH /api/admin/devices/:id	routes/admin.js
Read audit log	Settings.js	GET /api/admin/audit	models/AuditLog.js
System status	Settings.js	GET /api/admin/system	faceEngine/liveness status
Sign out	AdminLayout.js	POST /api/auth/logout	token revoked (tokenVersion+1)

8. Folder map

```
facedetection/
├─ README.md                  setup + security + testing guide
├─ docs/AUDIT.md              audit findings + every change made
├─ backend/
│  ├─ server.js               START HERE: boots everything
│  ├─ .env / .env.example     settings and secrets (never commit .env)
│  ├─ models/antispoof/       MiniFASNetV2.onnx, MiniFASNetV1SE.onnx
│  ├─ scripts/                create-admin, evaluate, rotate-keys, ...
│  ├─ test/                   58 automated tests
│  └─ src/
│     ├─ app.js               Express app: middleware + routes
│     ├─ db.js                MongoDB connection
│     ├─ config/index.js      reads + validates .env
│     ├─ middleware/          auth, security(CSRF), upload, validate, rateLimits, errorHandler
│     ├─ routes/              auth, kiosk, students, attendance, admin, schemas
│     ├─ services/            face AI + business logic (see section 9)
│     ├─ models/              Student, Attendance, Admin, Device, AuditLog
│     ├─ migrations/          upgrades old data automatically
│     └─ utils/               logger, errors, time, biometricCrypto
└─ frontend/
   ├─ public/                 index.html, favicon.svg
   └─ src/
      ├─ index.js, App.js     entry + all routes (URLs)
      ├─ api/client.js        every call to the backend
      ├─ auth/AuthContext.js  who is signed in, RequireAdmin
      ├─ hooks/useCamera.js   camera start/stop/capture
      ├─ components/          ui.js (icons, buttons, dialogs), FaceCapture.js
      ├─ pages/               Home, Login, Kiosk
      ├─ pages/admin/         AdminLayout, Dashboard, Students, StudentDetail,
      │                       RegisterStudent, Attendance, Settings
      ├─ utils/format.js      dates + form validation
      └─ styles/              app.css, kiosk.css (+ index.css tokens)
```

9. Backend modules — what it does & where it is used

Core

Module	What it does (short)	Used by
server.js	Entry point. Boot order: config → DB → migrate → AI models → gallery → listen.	`npm start`
src/app.js	Builds Express app: helmet, CORS, JSON limit, cookies, CSRF, auth, routes, error handler, serves React build.	server.js, all API tests
src/config/index.js	Reads `.env`, checks every value (zod), gives defaults. Only place that reads env.	Almost every module
src/db.js	Connects to MongoDB, logs disconnect/reconnect.	server.js, create-admin, tests
src/migrations/index.js	Upgrades old data (plain embeddings → encrypted samples; old attendance → dayKey). Builds indexes. Safe to run every start.	server.js, tests

Middleware (runs before routes)

Module	What it does	Used by
middleware/auth.js	Reads session cookies. Sets `req.admin` / `req.device`. `requireAdmin`, `requireKioskOrAdmin`, issue/clear cookies.	app.js, all routes
middleware/security.js	CSRF guard: POST/PATCH/PUT/DELETE need header `X-Requested-With` + allowed Origin.	app.js
middleware/upload.js	Multer: images kept in memory only, max 2 MB, JPEG/PNG only.	routes/kiosk.js, routes/students.js
middleware/validate.js	Checks body/query/params with zod → `req.valid`.	all routes
middleware/rateLimits.js	Limits: login 20 / 15 min, scan 600 / min, API 600 / min per IP.	app.js, routes/auth.js, routes/kiosk.js
middleware/errorHandler.js	Turns any error into a friendly JSON message; logs details + request id.	app.js

Routes (API endpoints — details in section 12)

Module	What it does	Mounted at
routes/auth.js	Login, logout, me, change password, activate/deactivate kiosk. Account lockout.	/api/auth
routes/kiosk.js	Receives camera frames, runs the scan session, returns state.	/api/kiosk
routes/students.js	Check sample, register, list, detail, edit, re-register face, delete, duplicates.	/api/students
routes/attendance.js	List/filter, edit status, delete, Excel export.	/api/attendance (+ /api/download-attendance)
routes/admin.js	Dashboard numbers, devices, audit log, system status.	/api/admin
routes/schemas.js	Shared rules: name, roll no, id, date, paging, safe regex.	students, attendance, admin routes

9. Backend modules (continued)

Services — face AI & business logic

Module	What it does	Used by
services/imageDecoder.js	Checks file really is JPEG/PNG (magic bytes), size limits, decodes to pixels.	faceEngine.js
services/faceEngine.js	Loads face-api models (WASM/TensorFlow). `detect`, `descriptor` (128 numbers), `pixelStats` (blur/light), `antispoofInput`. One-at-a-time queue, frees memory.	faceAnalysis, server.js, admin route, scripts
services/faceGeometry.js	From 68 landmarks: head turn (yaw), tilt (roll), up/down (pitch), eye openness (EAR); box overlap (IoU).	faceEngine, scanSession
services/quality.js	Face quality rules + user messages; pose check for registration (front/left/right).	faceAnalysis, enrollmentService
services/liveness.js	Passive anti-spoof (2 ONNX models) + `Challenge` (turn left/right, blink).	faceAnalysis, scanSession, server.js
services/faceAnalysis.js	One pass per frame: detect → count → quality → liveness → descriptor (skips expensive steps when not needed).	routes/kiosk.js, enrollmentService, evaluate script
services/matcher.js	Pure maths: distance, match with threshold + margin, find similar, centroid.	gallery, enrollmentService, scanSession, evaluate
services/gallery.js	Keeps all active templates decrypted in memory for fast matching; duplicate finder.	server.js, kiosk/students/admin routes
services/scanSession.js	Kiosk state machine (SEARCH → CHALLENGE → CONFIRM → DONE) + session store.	routes/kiosk.js
services/enrollmentService.js	Validates registration photos, builds templates, checks same person.	routes/students.js
services/attendanceService.js	One check-in per day, Late rule, cooldown, handles duplicate safely.	routes/kiosk.js
services/audit.js	Writes audit events (who did what, when).	all routes

Models (MongoDB)

Module	Stores	Used by
models/Student.js	Person + encrypted face samples (hidden by default).	students/admin routes, gallery, migrations
models/Attendance.js	Check-ins (unique per person per day).	attendanceService, attendance/admin/students routes
models/Admin.js	Admin login (bcrypt hash, lockout, tokenVersion).	auth middleware/route, create-admin, server.js
models/Device.js	Activated kiosks.	auth, kiosk, admin routes
models/AuditLog.js	Audit trail.	audit.js, admin route

Utils

Module	What it does	Used by
utils/logger.js	JSON logs; hides passwords, tokens, face data automatically.	most modules
utils/errors.js	AppError + helpers (badRequest, unauthorized, notFound, conflict).	middleware, routes, services
utils/time.js	Day key (YYYY-MM-DD) in APP_TIMEZONE, Late check.	attendanceService, routes, migrations
utils/biometricCrypto.js	AES-256-GCM encrypt/decrypt of face templates.	students route, gallery, migrations

9. Backend scripts (run with npm)

Command	File	Purpose
npm run create-admin -- admin	scripts/create-admin.js	Create admin / reset password
npm run models	scripts/download-models.js	Download + verify anti-spoof models
npm run evaluate -- <dataset>	scripts/evaluate.js	Accuracy report: FAR, FRR, TAR, threshold
npm run evaluate-liveness -- ...	scripts/evaluate-liveness.js	Spoof report: APCER, BPCER
npm run probe -- <folder>	scripts/probe-images.js	Quality/liveness numbers per image
npm run rotate-keys	scripts/rotate-keys.js	New secrets, re-encrypt templates

10. Frontend modules — what it does & where it is used

Module	What it does	Used by
src/index.js	Starts React, loads CSS.	browser
src/App.js	All URLs (routes) + redirects from old URLs (/register, /attendance, /records).	index.js, tests
api/client.js	Every backend call (fetch). Adds CSRF header, sends cookies, turns errors into friendly messages.	all pages, AuthContext, FaceCapture
auth/AuthContext.js	Knows who is signed in (admin / kiosk). login, logout, `RequireAdmin` guard.	App.js, Login, Kiosk, AdminLayout
hooks/useCamera.js	Starts camera, friendly camera errors, captures un-mirrored JPEG frames.	Kiosk.js, FaceCapture.js
components/ui.js	Icon, Spinner, Notice, Pill, EmptyState, Pagination, Modal, ConfirmDialog.	all pages
components/FaceCapture.js	Guided 5-pose capture with live server checks and auto-capture.	RegisterStudent, StudentDetail
utils/format.js	Date/time formatting in server time zone; name/ID validation.	admin pages
pages/Home.js	Landing: Kiosk or Admin console.	route /
pages/Login.js	Admin sign-in form.	route /login
pages/Kiosk.js	Camera screen: scan loop, face box, status, challenge arrow, result card, clock, device activation.	route /kiosk
pages/admin/AdminLayout.js	Sidebar menu, sign out, loads system info (time zone).	route /admin/*
pages/admin/Dashboard.js	Active / present / late / absent, today's check-ins, last 7 days, warnings.	/admin
pages/admin/Students.js	User list: search, status filter, re-register tab, duplicates tab, paging.	/admin/students
pages/admin/StudentDetail.js	Edit, activate/deactivate, re-register face, delete (confirm), recent attendance.	/admin/students/:id
pages/admin/RegisterStudent.js	Wizard: details → capture → review → done; duplicate warning.	/admin/students/new
pages/admin/Attendance.js	Records: date/search/status filters, edit status, delete, export Excel.	/admin/attendance
pages/admin/Settings.js	Change password, kiosk devices, audit log, system status.	/admin/settings
styles/app.css, kiosk.css, index.css	Design: colours, layout, tables, kiosk dark theme, mobile.	index.js, Kiosk.js
App.test.js, setupTests.js	9 UI tests (routing, login error, guards, API client, validation).	`npm test`

11. Libraries / packages used

Backend (backend/package.json)

Package	Why	Used in
express	Web server / API	app.js, routes
mongoose	MongoDB models + queries	models, db.js, routes, services
@vladmandic/face-api	Face detection, 68 landmarks, 128-d descriptor	faceEngine.js
@tensorflow/tfjs + tfjs-backend-wasm	Runs the face models fast on CPU (WASM)	faceEngine.js
onnxruntime-node	Runs anti-spoof ONNX models	liveness.js
jpeg-js, pngjs	Safe pure-JS image decoding	imageDecoder.js
multer	Receive uploaded frames (memory)	middleware/upload.js
zod	Validate .env and every request	config, validate, routes
jsonwebtoken	Signed session tokens	middleware/auth.js
bcryptjs	Password hashing	routes/auth.js, create-admin
cookie-parser	Read session cookies	app.js
helmet	Security headers + CSP	app.js
cors	Allowed browser origins	app.js
express-rate-limit	Brute-force / flood protection	rateLimits.js
exceljs	Excel export	routes/attendance.js
dotenv	Load .env file	config/index.js
supertest (dev), nodemon (dev)	API tests; auto-restart in dev	test/, `npm run dev`

Frontend (frontend/package.json)

Package	Why	Used in
react, react-dom	UI	everywhere
react-router-dom	Pages / URLs	App.js, pages
react-scripts	Dev server, build, tests (Create React App)	npm start / build / test
@testing-library/*	UI tests	App.test.js

Browser features (no library): `getUserMedia` for the camera, `fetch` for API calls, canvas for frame capture.

AI models

Model	Job	File / source
SSD MobileNet v1	Find faces	inside @vladmandic/face-api/model
Face Landmark 68	68 points (eyes, nose, jaw) → pose, alignment	inside @vladmandic/face-api/model
Face Recognition (ResNet)	128-number face template	inside @vladmandic/face-api/model
MiniFASNetV2 + MiniFASNetV1SE	Real face vs photo/screen	backend/models/antispoof (Silent-Face, Apache-2.0)

12. API endpoints

All under `/api`. "Admin" = signed-in admin cookie. "Kiosk" = activated device cookie. Non-GET calls need header `X-Requested-With: XMLHttpRequest`.

Method + path	Who	Does
GET /health	anyone	Server alive check
POST /auth/login	anyone	Sign in (sets cookie)
POST /auth/logout	anyone	Sign out + revoke token
GET /auth/me	anyone	Who am I (admin / kiosk / none)
POST /auth/change-password	Admin	Change password
POST /auth/kiosk/activate	Admin	Make this browser a kiosk
POST /auth/kiosk/deactivate	Kiosk	Remove kiosk mode
POST /kiosk/scan	Kiosk or Admin	Send one frame (+ sessionId) → state, message, face box, result
POST /students/check-sample	Admin	Check one registration frame (nothing saved)
POST /students	Admin	Register (name, rollNo, poses, 3–8 images)
GET /students	Admin	List (search, status, needsReenroll, page)
GET /students/duplicates	Admin	Pairs of similar faces
GET /students/:id	Admin	Details + recent attendance
PATCH /students/:id	Admin	Edit name / rollNo / active
PUT /students/:id/face	Admin	Replace face samples
DELETE /students/:id	Admin	Delete person (history kept)
GET /attendance	Admin	List (from, to, search, status, studentId, page)
GET /attendance/export	Admin	Excel file
PATCH /attendance/:id	Admin	Set Present / Late
DELETE /attendance/:id	Admin	Delete record
GET /download-attendance	Admin	Old URL: today's Excel
GET /admin/dashboard	Admin	Dashboard numbers
GET /admin/devices, PATCH /admin/devices/:id	Admin	List / revoke kiosks
GET /admin/audit	Admin	Audit log (category, page)
GET /admin/system	Admin	Engine, liveness, thresholds, time zone

Kiosk scan states (returned by POST /kiosk/scan)

NO_FACE · QUALITY · MULTIPLE_FACES · VERIFYING · CHECKING_LIVENESS · CHALLENGE · MARKED · ALREADY_MARKED · UNKNOWN · UNVERIFIED · SPOOF · LIVENESS_FAILED · TIMEOUT · ERROR

13. Database collections (MongoDB: faceattendance)

Collection	Main fields	Indexes / rules
students	name, rollNo, active, samples[] (encrypted template, pose, quality, liveness), sampleCount, hasLegacySample, createdAt	rollNo unique · samples hidden from queries
attendances	studentId, studentName, rollNo (snapshot), dayKey, checkInAt (UTC), status (Present/Late), source, verification {distance, confidence, liveness, challenge}, deviceId	**(studentId + dayKey) unique** = no duplicates
admins	username, passwordHash (bcrypt), role, active, tokenVersion, failedLogins, lockUntil, lastLoginAt	username unique
devices	name, active, createdBy, lastSeenAt	—
auditlogs	action, category (admin/auth/security), actor, target, details, ip, createdAt	createdAt, category

Never stored: face photos, raw camera frames, plain passwords, unencrypted face templates.

14. Important settings (backend/.env)

Setting	Default	Meaning (easy words)
MONGO_URI	mongodb://127.0.0.1:27017/faceattendance	Database address
JWT_SECRET	(secret)	Signs login cookies. Change = everyone logged out
BIOMETRIC_KEY	(secret)	Encrypts face templates. **Back it up!** Lost = re-register all
APP_TIMEZONE	system (Asia/Kolkata)	When a new day starts
LATE_AFTER	empty	e.g. 09:30 → later check-ins = Late
RECOGNITION_THRESHOLD	0.5	Max face distance to accept (lower = stricter)
HIGH_CONFIDENCE_THRESHOLD	0.4	Very close match = fewer frames needed
RECOGNITION_MIN_MARGIN	0.06	Must beat 2nd-best person by this much
STABLE_FRAMES	3	Same person on this many frames
DUPLICATE_FACE_THRESHOLD	0.42	Registration warning if closer than this
LIVENESS_THRESHOLD	0.6	Anti-photo score needed (0–1)
CHALLENGE_TYPES	turn_left,turn_right	Which challenges to ask
CHALLENGE_TURN_DELTA / TIMEOUT_MS	0.15 / 10000	How much to turn / time allowed
MIN_FACE_SIZE_PX	90	Smaller face = "Move closer"
MIN_SHARPNESS / MIN_BRIGHTNESS / MAX_BRIGHTNESS	20 / 55 / 205	Blur and lighting limits
ATTENDANCE_COOLDOWN_SEC	60	Ignore repeat scans of same person
ADMIN_SESSION_HOURS / KIOSK_SESSION_DAYS	8 / 30	How long logins last

Full list with comments: `backend/.env.example`. Measure thresholds with `npm run evaluate` before real use.

15. Security in short

Login

- bcrypt password hash
- httpOnly + SameSite=Strict cookie (8 h)
- 5 wrong tries → locked 15 min
- Logout revokes the token on the server

Kiosk

- Own device token, only for scanning
- Cannot open admin pages
- Revoke anytime in Settings

Requests

- CSRF header + Origin check
- zod validation (no NoSQL injection)
- Rate limits, 20 kB JSON limit
- Helmet headers + CSP

Uploads

- Memory only, never saved to disk
- Magic-byte check (real JPEG/PNG)
- Size + resolution limits

Biometric data

- AES-256-GCM encrypted templates
- Never sent to the browser
- No face photos stored

Logs & audit

- Passwords/tokens/face data auto-hidden
- Audit log: register, delete, export, overrides, failed logins, spoof attempts

Limits (be honest): one normal webcam cannot stop 100% of attacks (e.g. high-quality 3D masks or hacked video input). It stops printed photos, phone/monitor screens and simple replays.

16. Messages & troubleshooting

You see	Why	Do this
Face not recognized.	Not registered, or too different from samples	Register / re-register the person
Unable to verify identity.	Two registered people look alike (often duplicates)	Users → Possible duplicates → delete extras
Unable to confirm a live person.	Anti-photo score low (photo/screen, or bad light)	Real face, better light, no glare
Liveness check was not completed.	Head-turn not done in time	Turn a little to the named side, then look back
Move closer / Move back a little.	Face too small / too big	Stand ~50–80 cm from camera
Lighting is too low / too bright.	Dark room / light behind or on face	Face the light, avoid backlight
Please ensure only one person is visible.	2+ faces	One person at a time
Unable to access the camera...	Permission denied / camera busy	Allow camera in browser; close other camera apps
This device is not activated	No kiosk token	Admin signs in on kiosk → Activate
Invalid username or password.	Wrong login	`npm run create-admin -- admin` resets it
Port 5000/3000 already in use	Server already running	Close the other terminal / process
Startup failed: Invalid configuration	Bad/missing .env value	Compare with .env.example
Could not decrypt biometric samples	BIOMETRIC_KEY changed/lost	Restore .env backup, else re-register

17. Commands cheat-sheet

Folder	Command	Does
backend	npm start	Run API (port 5000)
backend	npm run dev	Run with auto-restart (nodemon)
backend	npm test	58 backend tests
backend	npm run create-admin -- admin	Create / reset admin
backend	npm run evaluate -- <dataset>	Accuracy report
backend	npm run evaluate-liveness -- --live <dir> --spoof <dir>	Anti-spoof report
backend	npm run probe -- <folder>	Quality numbers per photo
backend	npm run rotate-keys	New secrets + re-encrypt
backend	npm run models	Verify anti-spoof models
frontend	npm start	Run website (port 3000)
frontend	npm test	9 UI tests
frontend	npm run build	Production build (served by backend)

Testing checklist (manual, on the real camera)

- Registered person → marked in ~3–5 s; again → "already marked"
- Unregistered person → "Face not recognized", nothing saved
- Printed photo / phone photo / video → never marked
- Two people in view → "only one person"
- Stand 30 s in front of camera → still exactly one record
- Dark room, glasses, far/near → guidance, never a wrong name

18. Remember it in 60 seconds

1 sentence: React kiosk sends frames → Express backend runs **Detect** → **Quality** → **Liveness** → **Embed** → **Match** → **Challenge** → **Confirm** → MongoDB saves **one record per person per day**.

3 folders to know

- `backend/src/services` = the brain
- `backend/src/routes` = the doors (API)
- `frontend/src/pages` = the screens

3 most important files

- `scanSession.js` = kiosk decision
- `matcher.js` = who is it?
- `liveness.js` = is it real?

3 numbers

- **0.5** max face distance
- **0.06** margin over 2nd best
- **3** stable frames

3 secrets

- `.env` never in git
- BIOMETRIC_KEY = back up
- No face photos stored

Memory trick for the kiosk flow: "L-Q-V-C-C-M"

Look (find face) → **Q**uality → **V**erify (liveness + identity) → **C**hallenge (turn head) → **C**onfirm → **M**ark.

Memory trick for registration: "D-5-R-S"

Details → **5** guided photos → **R**eview (duplicate check) → **S**ave encrypted.

More detail: README.md (setup, security, testing) and docs/AUDIT.md (audit findings and all changes).